

Clase 5 extra

## SOBRE EL BUFFER

Read(...) – Readln(...)

1

### Buffer: Porción de memoria interna

- Es una porción de memoria que se usa para almacenar la información que va ingresando el usuario. No se puede acceder directamente. La accedemos mediante las ejecuciones de READ o READLN.
- Cada vez que ejecutamos una instrucción READ, se busca en el buffer el dato necesario.
- Si el READ(DATO) es de un DATO de tipo INTEGER, se buscará en el buffer un número ENTERO.
- Si el READ(DATO) es de un DATO de tipo CHAR, se buscará un caracter.
- Como se ponen cosas en el buffer??? Cada vez que el usuario tiene delante suyo la CONSOLA con el cursor esperando una entrada, ingresa texto (ya sea números o símbolos) y hace ENTER.

2

### Ejemplo 1

- El programa ejecuta la instrucción 1:

```
Program manejoBUFFER;
Var ALGO: char;
```

```
begin
  Writeln('INGRESE ALGO');
  Read(ALGO); {le pido que me de un CHAR del buffer}
  Writeln('Ha ingresado lo siguiente: ', ALGO);
```

```
Readln; {leerá hasta el ENTER que había hecho el usuario}
Readln;
End.
```

INGRESE ALGO:

Buffer

Al comenzar el programa, el buffer esta vacío.

3

### Ejemplo 1

- El programa ejecuta la **instrucción 2**:

```
Program manejoBUFFER;
Var ALGO: char;
```

```
begin
  Writeln('INGRESE ALGO');
  Read(ALGO); {le pido que me de un CHAR del buffer}
  Writeln('Ha ingresado lo siguiente: ', ALGO);
```

```
Readln; {leerá hasta el ENTER que había hecho el usuario}
Readln;
End.
```

READ. SI EL BUFFER ESTA VACIO, queda esperando a que el usuario escriba y haga ENTER (el cursor titila)

INGRESE ALGO:

10 casas tiene mi barrio.

Buffer

10 casas tiene mi barrio.ENTER

4

### Ejemplo 1

- El programa ejecuta la **instrucción 2**:

```
Program manejoBUFFER;
Var ALGO: char;
```

```
begin
  Writeln('INGRESE ALGO');
  Read(ALGO); {le pido que me de un CHAR del buffer}
  Writeln('Ha ingresado lo siguiente: ', ALGO);
```

```
Readln;
Readln;
End.
```

INGRESE ALGO:

10 casas tiene mi barrio.

Buffer

10 casas tiene mi barrio.ENTER

READ. Toma el primer elemento del BUFFER (que ahora si tiene contenido). Busca un elemento del tipo del dato que hay en el READ. ALGO es de tipo CHAR. En este caso el primer CHAR es el '1'. ALGO se inicializa con el caracter '1'.

5

### Ejemplo 1

- El programa ejecuta la **instrucción 2**:

```
Program manejoBUFFER;
Var ALGO: char;
```

```
begin
  Writeln('INGRESE ALGO');
  Read(ALGO); {le pido que me de un CHAR del buffer}
  Writeln('Ha ingresado lo siguiente: ', ALGO);
```

```
Readln;
Readln;
End.
```

INGRESE ALGO:

10 casas tiene mi barrio.

Buffer

10 casas tiene mi barrio.ENTER

READ. Toma el primer elemento del BUFFER (que ahora si tiene contenido). Busca un elemento del tipo del dato que hay en el READ. ALGO es de tipo CHAR. En este caso el primer CHAR es el '1'. ALGO se inicializa con el caracter '1'.

6

## Ejemplo 1

- El programa ejecuta la instrucción 3:

```
Program manejoBUFFER;
Var ALGO: char;
```

```
begin
  Writeln('INGRESE ALGO');
  Read(ALGO); {le pido que me de un CHAR del buffer}
  Writeln('Ha ingresado lo siguiente: ', ALGO);
```

```
  Readln;
  Readln;
End.
```

INGRESE ALGO:  
10 casas tiene mi barrio.  
Ha ingresado lo siguiente: 1

ALGO  
'1'

Buffer  
0 casas tiene mi barrio. ENTER

Observen que una vez que termina el READ, desaparece ese primer caracter que acaba de leer; se saca del buffer.

7

## Ejemplo 1

- El programa ejecuta la instrucción 3:

```
Program manejoBUFFER;
Var ALGO: char;
```

```
begin
  Writeln('INGRESE ALGO');
  Read(ALGO); {le pido que me de un CHAR del buffer}
  Writeln('Ha ingresado lo siguiente: ', ALGO);
```

```
  Readln;
  Readln;
End.
```

INGRESE ALGO:  
10 casas tiene mi barrio.  
Ha ingresado lo siguiente: 1

ALGO  
'1'

Buffer  
0 casas tiene mi barrio. ENTER

ES IMPORTANTE ENTENDER QUE ESE '1' ES UN CARÁCTER, O SEA QUE NO PUEDO HACER OPERACIONES MATEMÁTICAS CON ESE DATO.

8

## Ejemplo 1

- El programa ejecuta la instrucción 3:

```
Program manejoBUFFER;
Var ALGO: char;
```

```
begin
  Writeln('INGRESE ALGO');
  Read(ALGO); {le pido que me de un CHAR del buffer}
  Writeln('Ha ingresado lo siguiente: ', ALGO);
```

```
  Readln; {leerá hasta el ENTER que había hecho el usuario}
  Readln; {lee o espera el ENTER para finalizar}
End.
```

INGRESE ALGO:  
10 casas tiene mi barrio.  
Ha ingresado lo siguiente: 1

ALGO  
'1'

Buffer  
ENTER

9

## Ejemplo 1

- El programa ejecuta la instrucción 3:

```
Program manejoBUFFER;
Var ALGO: char;
```

```
begin
  Writeln('INGRESE ALGO');
  Read(ALGO); {le pido que me de un CHAR del buffer}
  Writeln('Ha ingresado lo siguiente: ', ALGO);
```

```
  Readln; {leerá hasta el ENTER que había hecho el usuario}
  Readln; {lee o espera el ENTER para finalizar}
End.
```

INGRESE ALGO:  
10 casas tiene mi barrio.  
Ha ingresado lo siguiente: 1

ALGO  
'1'

Buffer

10

## Ejemplo 1

- El programa ejecuta la instrucción 3:

```
Program manejoBUFFER;
Var ALGO: char;
```

```
begin
  Writeln('INGRESE ALGO');
  Read(ALGO); {le pido que me de un CHAR del buffer}
  Writeln('Ha ingresado lo siguiente: ', ALGO);
```

```
  Readln; {leerá hasta el ENTER que había hecho el usuario}
  Readln; {lee o espera el ENTER para finalizar}
End.
```

INGRESE ALGO:  
10 casas tiene mi barrio.  
Ha ingresado lo siguiente: 1

ALGO  
'1'

Buffer

11

## Ejemplo 2

- El programa ejecuta la primera instrucción:

```
Program manejoBUFFER;
Var ALGO: integer;
```

```
begin
  Writeln('INGRESE ALGO');
  Read(ALGO); {le pido que me de un INTEGER del buffer}
  Writeln('Ha ingresado lo siguiente: ', ALGO);
```

```
  Readln; {leerá hasta el ENTER que había hecho el usuario}
  Readln;
End.
```

INGRESE ALGO:

ALGO

Buffer

12

## Ejemplo 2

- El programa ejecuta la primera instrucción:

```
Program manejoBUFFER;
Var ALGO: integer;
```

```
begin
  Writeln('INGRESE ALGO');
  Read(ALGO); {le pido que me de un INTEGER del buffer}
  Writeln('Ha ingresado lo siguiente: ', ALGO);

  Readln; {leerá hasta el ENTER que había hecho el usuario}
  Readln;
End.
```

READ. SI EL BUFFER ESTA VACIO, queda esperando a que el usuario escriba y haga ENTER (el cursor titila)

INGRESE ALGO:  
10 casas tiene mi barrio.

ALGO

Buffer  
10 casas tiene mi barrio. ENTER

13

## Ejemplo 2

- El programa ejecuta la primera instrucción:

```
Program manejoBUFFER;
Var ALGO: integer;
```

```
begin
  Writeln('INGRESE ALGO');
  Read(ALGO); {le pido que me de un INTEGER del buffer}
  Writeln('Ha ingresado lo siguiente: ', ALGO);

  Readln;
  Readln;
End.
```

INGRESE ALGO:  
10 casas tiene mi barrio.

ALGO

Buffer  
10 casas tiene mi barrio. ENTER

READ. Toma el primer elemento del BUFFER (que ahora si tiene contenido). Busca un elemento del tipo del dato que hay en el READ. ALGO es de tipo INTEGER. En este caso el primer INTEGER es el 10. ALGO se inicializa con el número 10.

14

## Ejemplo 2

- El programa ejecuta la primera instrucción:

```
Program manejoBUFFER;
Var ALGO: integer;
```

```
begin
  Writeln('INGRESE ALGO');
  Read(ALGO); {le pido que me de un INTEGER del buffer}
  Writeln('Ha ingresado lo siguiente: ', ALGO);
```

```
Readln;
Readln;
End.
```

INGRESE ALGO:  
10 casas tiene mi barrio.

ALGO  
10

Buffer  
10 casas tiene mi barrio. ENTER

READ. Toma el primer elemento del BUFFER (que ahora si tiene contenido). Busca un elemento del tipo del dato que hay en el READ. ALGO es de tipo INTEGER. En este caso el primer INTEGER es el 10. ALGO se inicializa con el número 10.

15

## Ejemplo 2

- El programa ejecuta la primera instrucción:

```
Program manejoBUFFER;
Var ALGO: integer;
```

```
begin
  Writeln('INGRESE ALGO');
  Read(ALGO); {le pido que me de un INTEGER del buffer}
  Writeln('Ha ingresado lo siguiente: ', ALGO);
```

```
Readln; {leerá hasta el ENTER que había hecho el usuario}
Readln;
End.
```

INGRESE ALGO:  
10 casas tiene mi barrio.  
Ha ingresado lo siguiente: 10

Buffer  
casas tiene mi barrio. ENTER

16

## Ejemplo 2

- El programa ejecuta la primera instrucción:

```
Program manejoBUFFER;
Var ALGO: integer;
```

```
begin
  Writeln('INGRESE ALGO');
  Read(ALGO); {le pido que me de un INTEGER del buffer}
  Writeln('Ha ingresado lo siguiente: ', ALGO);
```

```
Readln; {leerá hasta el ENTER que había hecho el usuario}
Readln;
End.
```

INGRESE ALGO:  
10 casas tiene mi barrio.  
Ha ingresado lo siguiente: 10

Buffer  
ENTER

17

## Ejemplo 2

- El programa ejecuta la primera instrucción:

```
Program manejoBUFFER;
Var ALGO: integer;
```

```
begin
  Writeln('INGRESE ALGO');
  Read(ALGO); {le pido que me de un INTEGER del buffer}
  Writeln('Ha ingresado lo siguiente: ', ALGO);
```

```
Readln; {leerá hasta el ENTER que había hecho el usuario}
Readln;
End.
```

INGRESE ALGO:  
10 casas tiene mi barrio.  
Ha ingresado lo siguiente: 10

Buffer

18

## Ejemplo 2

- El programa ejecuta la primera instrucción:

```
Program manejoBUFFER;
```

```
Var ALGO: integer;
```

```
begin
```

```
  Writeln('INGRESE ALGO');
```

```
  Read(ALGO); {le pido que me de un INTEGER del buffer}
```

```
  Writeln('Ha ingresado lo siguiente: ', ALGO);
```

```
  Readln; {leerá hasta el ENTER que había hecho el usuario}
```

```
  Readln;
```

```
End.
```

```
INGRESE ALGO:
10 casas tiene mi barrio.
Ha ingresado lo siguiente: 10
```

Buffer

19

COROLARIO: read(DATO) donde DATO es de tipo T

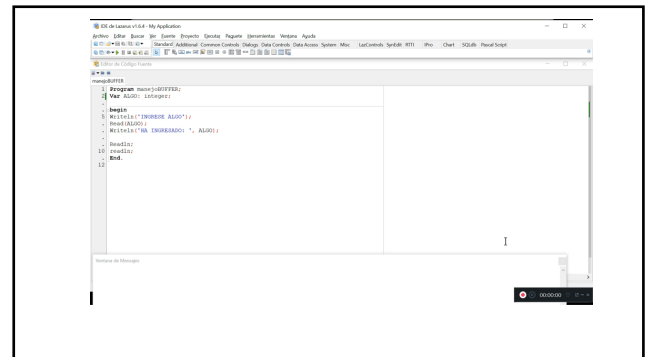
- El read **BUSCA en el BUFFER**.
- ¿Qué busca? Poder cargar en el DATO, un elemento del tipo T.
- Si le digo que DATO es INTEGER, buscará un entero. Esto es, buscará en el BUFFER DIGITOS (los dígitos son cada una de las cifras) hasta encontrar un **espacio** o un **ENTER**.
- Si le pido un **entero**, y el usuario ingresó un símbolo distinto de dígito (por ejemplo una A) se produce un **ERROR EN EJECUCION**. ¡También si el usuario ingresa un **NUMERO REAL**! Porque busca sólo dígitos hasta espacio o enter, al encontrar el punto o coma decimal, da **ERROR** en ejecución.
- Si en el BUFFER no hay nada, quedará esperando con el cursor titilando hasta que el usuario ingrese algo y haga ENTER.

20

## A continuación: Tres ejecuciones.

- La primera ejecución funciona bien.
- La segunda también. Luego de la lectura, las letras quedan en el buffer. Esto no representa ningún problema. Si hubiera otro read luego, se accedería a esas letras (rg).
- En la tercera ejecución se genera un error en EJECUCION por mal ingreso del dato por parte del usuario. Esto es incontrolable (por ahora). Podemos controlar con código que sea positivo, que sea par, que sea mayor o menor que otro, pero no podemos controlar ni evitar que no sea algo numérico.

21



22